Secure Software Development Lifecycle

Room Clarity uses a lightweight secure software development lifecycle appropriate for a beta Zoom app that processes meeting transcript data.

Scope

The SSDLC applies to application code, Zoom integration code, deployment configuration, dependencies, privacy-facing pages, and operational procedures for `https://roomclarity.com`.

Development Controls

- Changes are tracked in source control and reviewed before release.
- Product changes involving meeting transcripts, participant information, Zoom APIs, OAuth, RTMS, model providers, or dashboard access are treated as security-sensitive.
- Security-sensitive changes must identify affected data flows, credentials, access controls, logging behavior, and user-facing disclosures.
- Secrets are supplied through deployment environment variables or managed secret stores and are not committed to the repository.
- The service validates model output against structured constraints before incorporating it into meeting state.
- Transcript text is treated as untrusted input.

Security Checks

Before external beta release, the project runs:

- Dependency audit for production packages.
- SAST scan using CodeQL, Semgrep, or equivalent.
- DAST scan against the deployed HTTPS service using OWASP ZAP baseline or equivalent.
- Functional evals for transcript extraction behavior.
- Manual verification of security headers, webhook signature verification, access tokens, and admin route protections.

Release Controls

- Production deployments use Google Cloud Run and HTTPS.
- The public base URL is `https://roomclarity.com`.
- Required runtime secrets include Zoom OAuth credentials, Zoom webhook secret token, RTMS credentials if separate, model provider keys, and the Room Clarity admin token.
- Changes are verified after deployment using health checks, header checks, and targeted endpoint tests.